Machine Learning

Chapter 18: Optimization

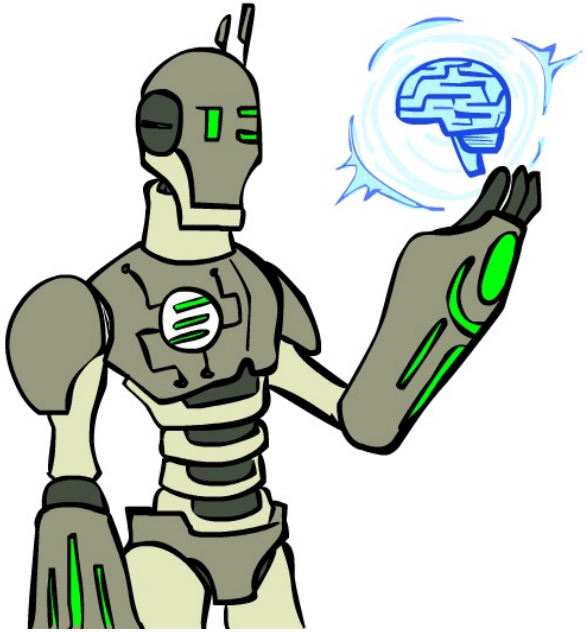
Spring 2025

Instructor: Xiaodong Gu
Presenter: Xijun Li



Today

- **Optimization**
- **Learning Mode**
- **Learning Rate**
- **Algorithms**
  - Adagrad, RMSProp, Adam,



Review: Optimization

- Find a θ to get the minimum loss function.

$$\theta^* = \arg\min_{\theta} L(\theta)$$

L: loss function

θ: model parameters

$$\theta^0 \rightarrow \theta^1 \rightarrow \theta^2 \rightarrow \cdots$$

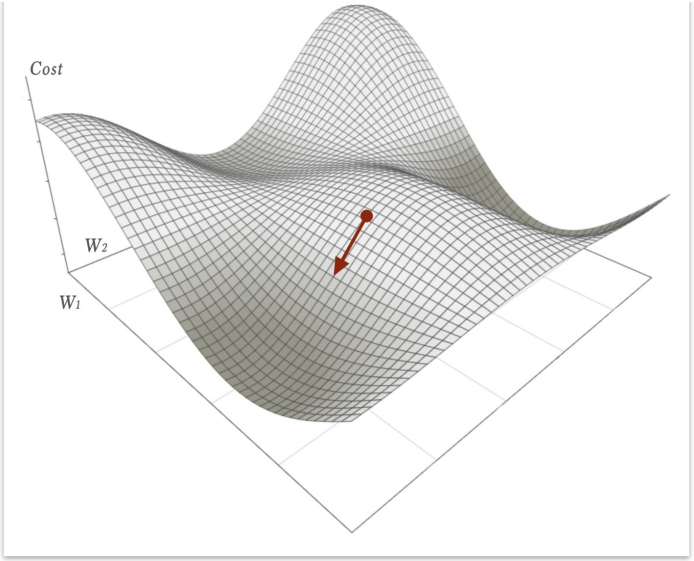
$$\theta^1 = \theta^0 - \eta \nabla L(\theta^0)$$

$$\theta^2 = \theta^1 - \eta \nabla L(\theta^1)$$

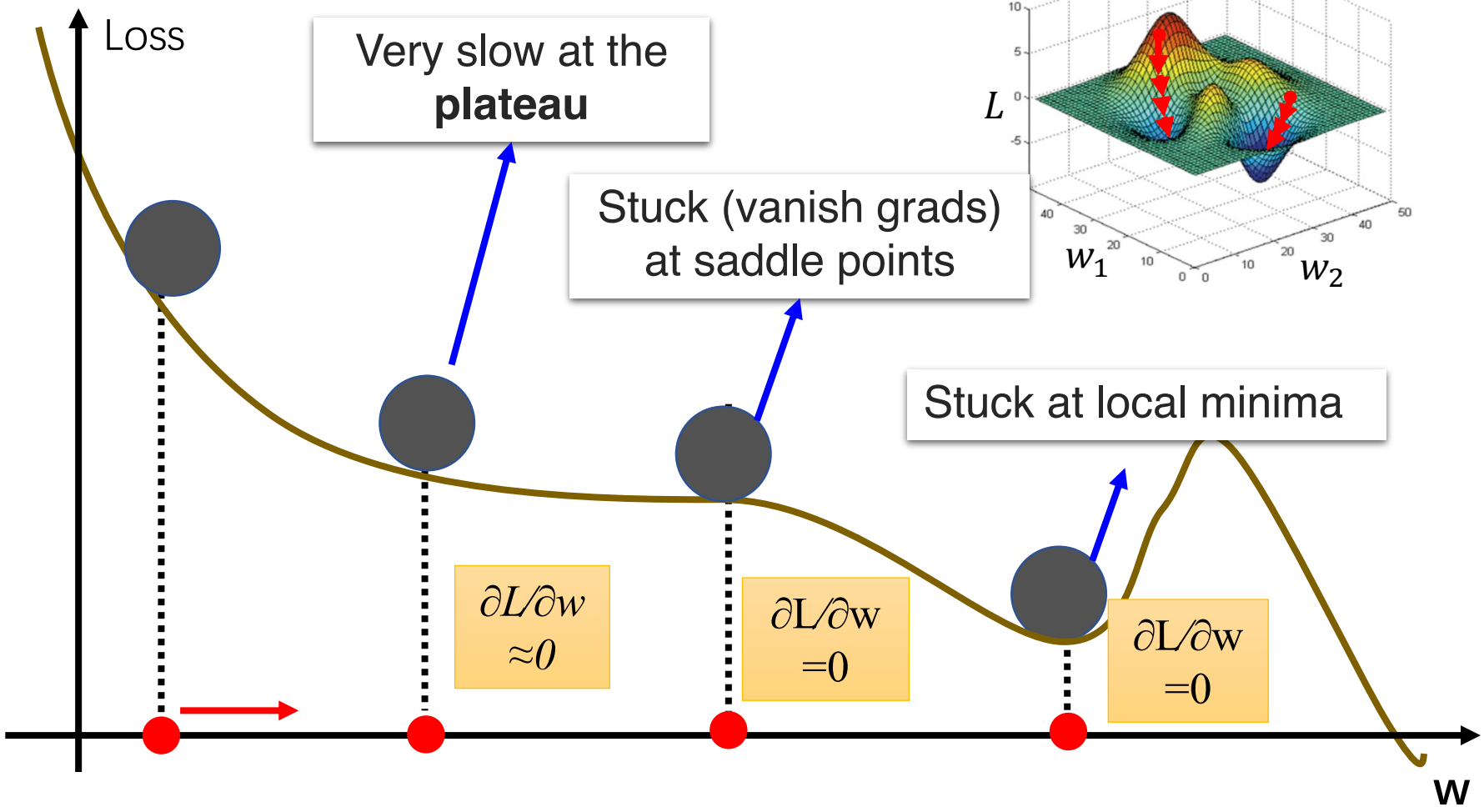
$$\vdots$$

$$\nabla L(\theta) = \begin{bmatrix} \partial L(\theta)/\partial w_1 \\ \partial L(\theta)/\partial w_2 \\ \vdots \end{bmatrix}$$

雅可比矩阵



Challenges of Gradient Descend

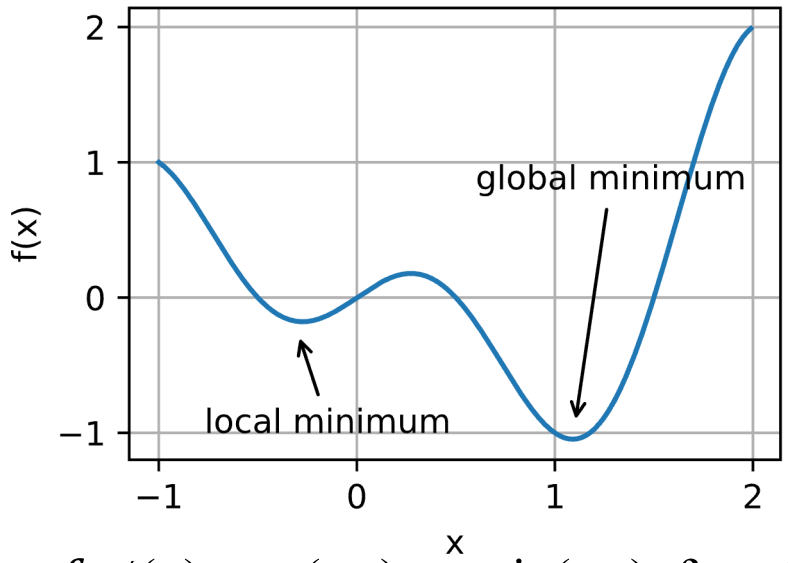


Challenge1: Local Minima

- The algorithm may only minimize the objective function locally, rather than globally

Example:

$$f(x)=x\cdot\cos(\pi x) \text{ for } -1.0\leq x\leq 2.0$$



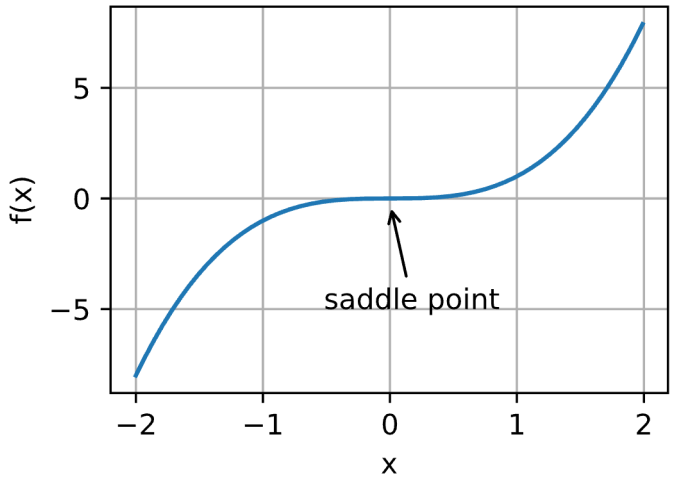
some degree of noise might knock the parameter out of the local minimum. In fact, this is one of the beneficial properties of stochastic gradient descent

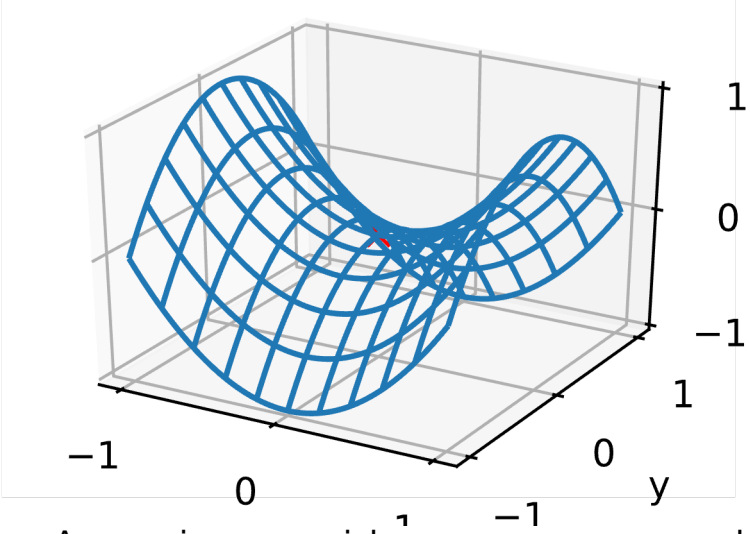
$$f\ '(x)=\cos(\pi x)-\pi x\cdot\sin(\pi x) \text{ for } -1.0\leq x\leq 2.0$$

Challenge2: Saddle Points

- A saddle point is a location where all gradients of a function vanish but which is neither a global nor a local minimum.

Example: $f(x)=x^3$ $f(x,y)= x^2-y^2$ at $(0,0)$





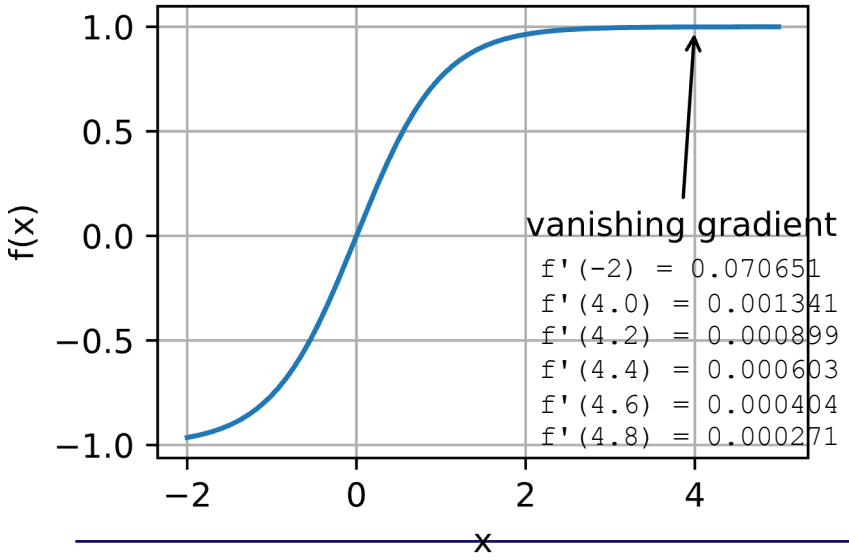
A maximum with respect to y and a minimum with respect to x

Challenge3: Vanishing Gradients

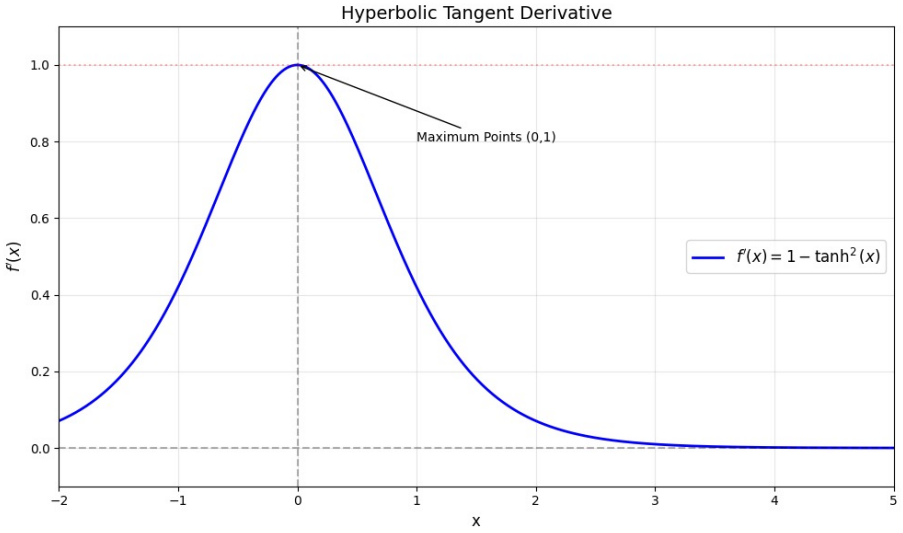
- At some points, the gradient of f is close to nil. Consequently optimization will get stuck for a long time before we make progress.

Example:

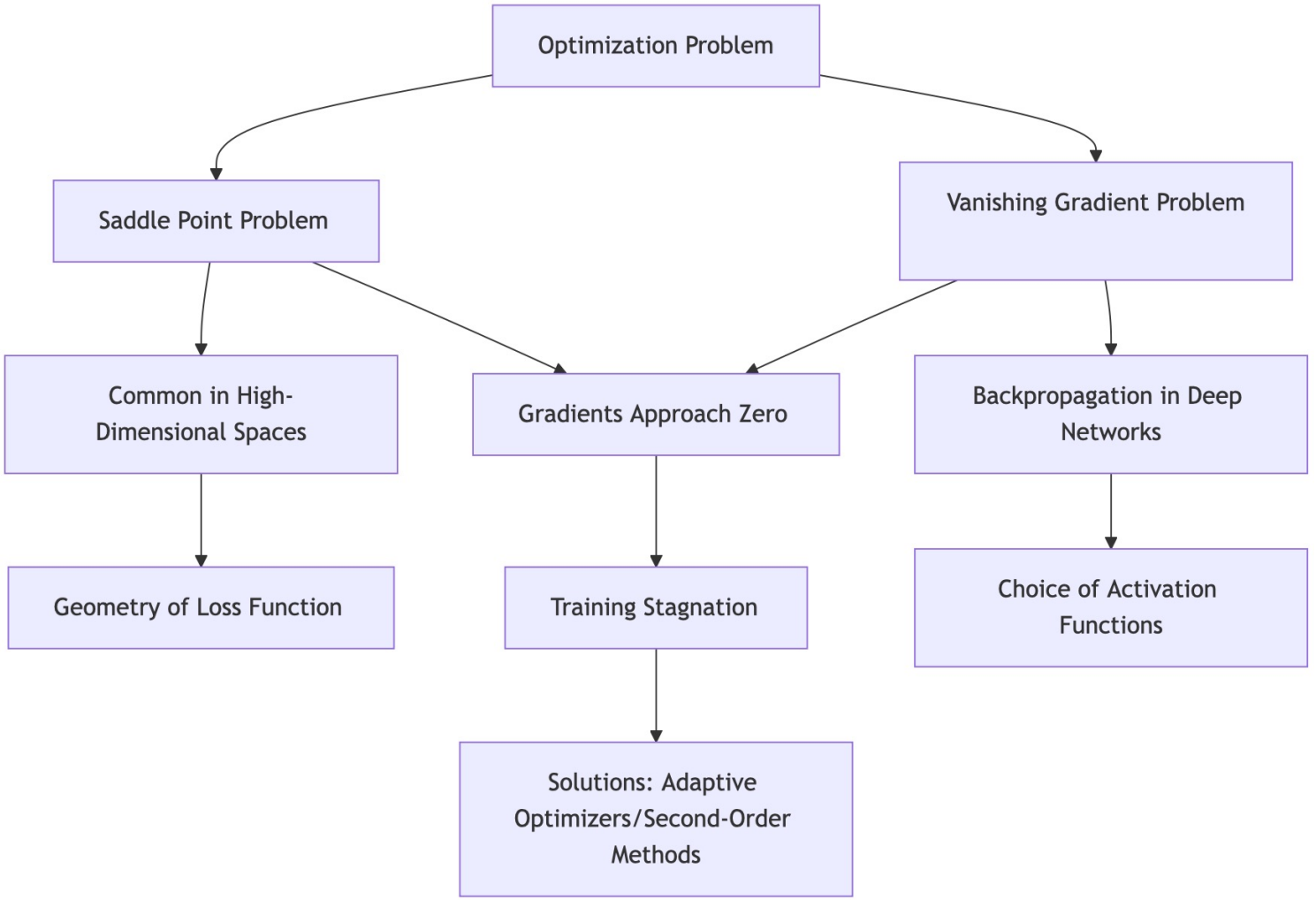
$f(x)=\tanh(x)$



$f'(x)=1-\tanh^2(x)$



The Relation between Saddle Points & Vanishing Gradient



The Relation between Saddle Points & Vanishing Gradient

Characteristic	Saddle Point Problem	Vanishing Gradient Problem
Root Cause	Geometry of high-dimensional loss	Chain rule accumulation in backprop
Location	Anywhere in parameter space	Early layers of deep networks
Math Nature	Hessian matrix has ± eigenvalues	Gradient norms decay exponentially
Typical Trigger	Non-convex optimization	Sigmoid/tanh activation functions
Parameter Impact	Global optimization blocked	Shallow parameters fail to learn

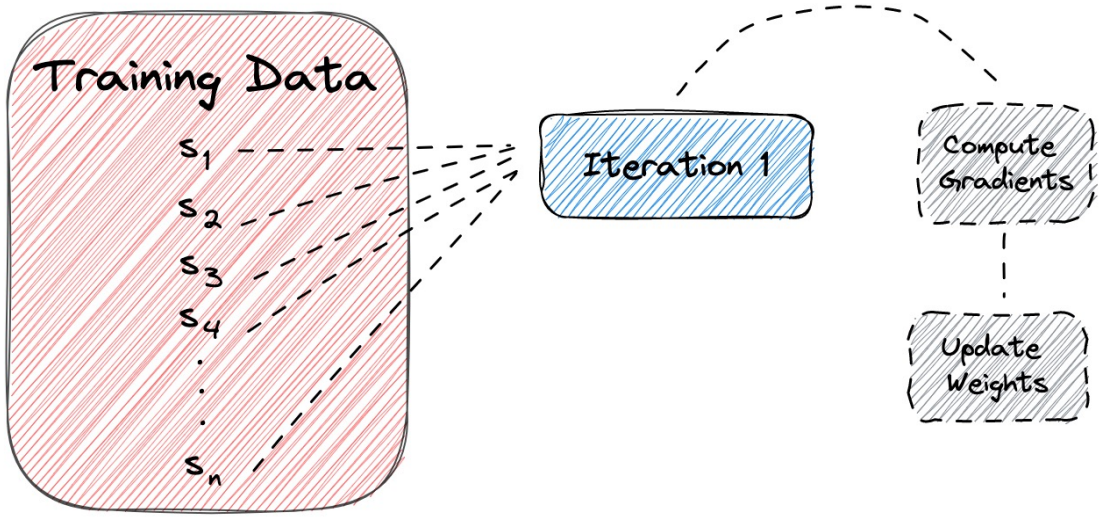


Learning Mode

Online Learning vs Offline Learning vs Minibatch Learning

Batch Gradient Descend

- **Batch Training**: computing the gradient over all of the training examples.

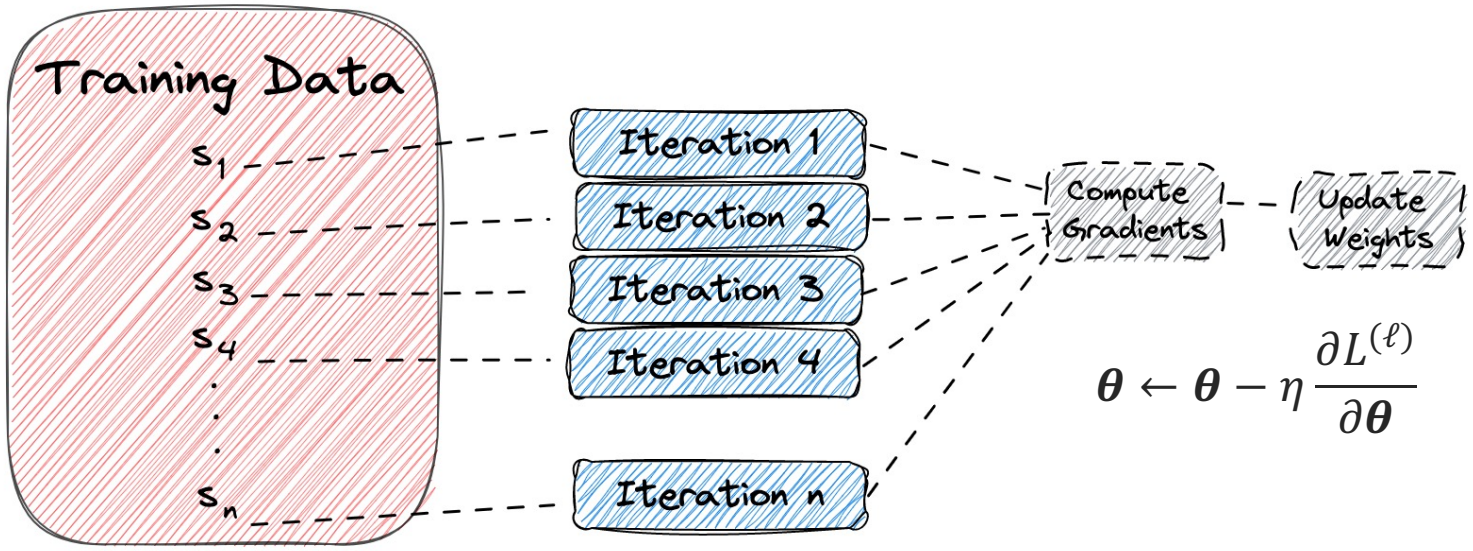


Compute loss function over the entire training set: $L(\theta) = \frac{1}{N}\sum_{\ell=1}^{N} L^{(\ell)}$

- Impractical if you have a large dataset!

Stochastic Gradient Descend (SGD)

- Update the parameters based on the gradient for a single training example:

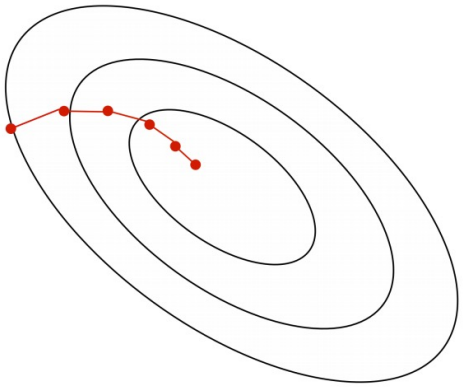


- Also known as online learning

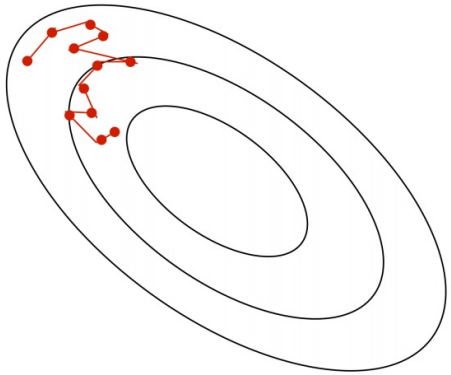
Problem: if we only look at one training example at a time, we can ‘t exploit efficient vectorized operations.

GD vs SGD

- Batch gradient descent moves directly downhill while SGD takes steps in a noisy direction, but moves downhill on average.



Batch gradient descent

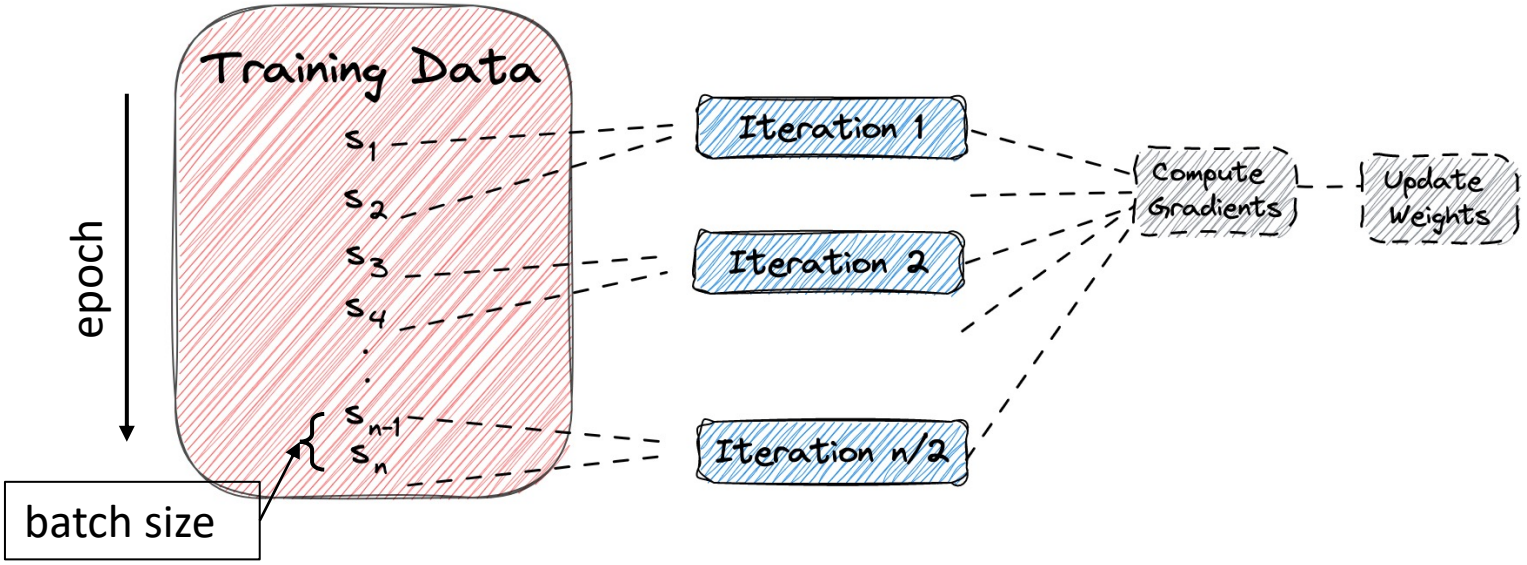


Stochastic gradient descent

- No need to store the whole sample.
- Can adapt to changes in sample distribution over time.
- Can adapt to physical changes in system components.

Mini-batch Gradient Descent

- **A compromise approach**: compute the gradients on a medium-sized set of training examples, called a mini-batch.



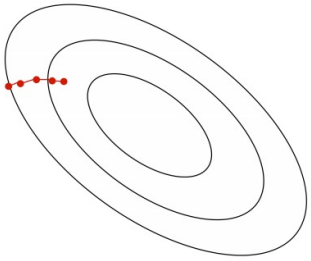
- can be seen as **sampled i.i.d. from the training set**.
- in practice, we typically go in order through the training set.
- each entire pass over the dataset is called an **epoch**.



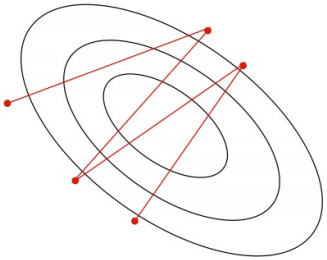
Learning Rate

Momentum, Adaptive LR

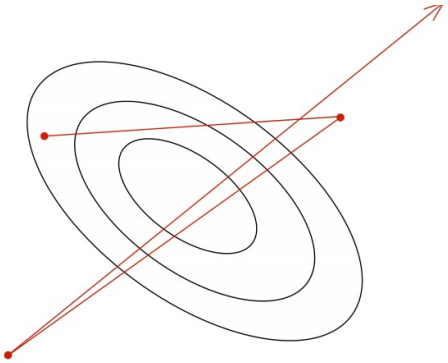
Learning Rate



η too small:
slow progress



η too large:
oscillations

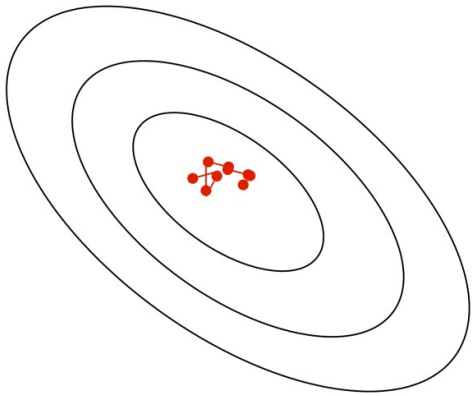


η much too large:
instability

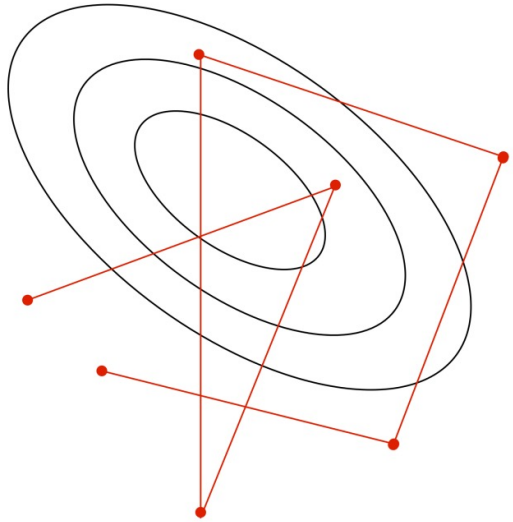
SGD Learning Rate

- In stochastic training, the learning rate also influences the fluctuations due to the stochasticity of the gradients.

small learning rate



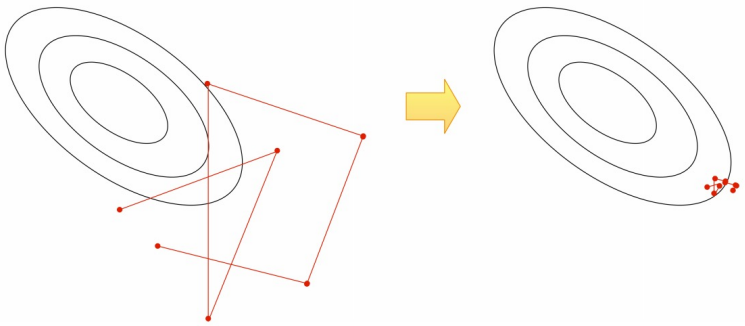
large learning rate

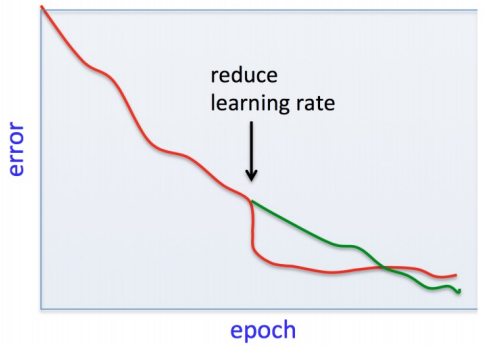


- Typical strategy:
  - Use a large learning rate early in training so you can get close to the optimum
  - Gradually decay the learning rate to reduce the fluctuations

SGD Learning Rate

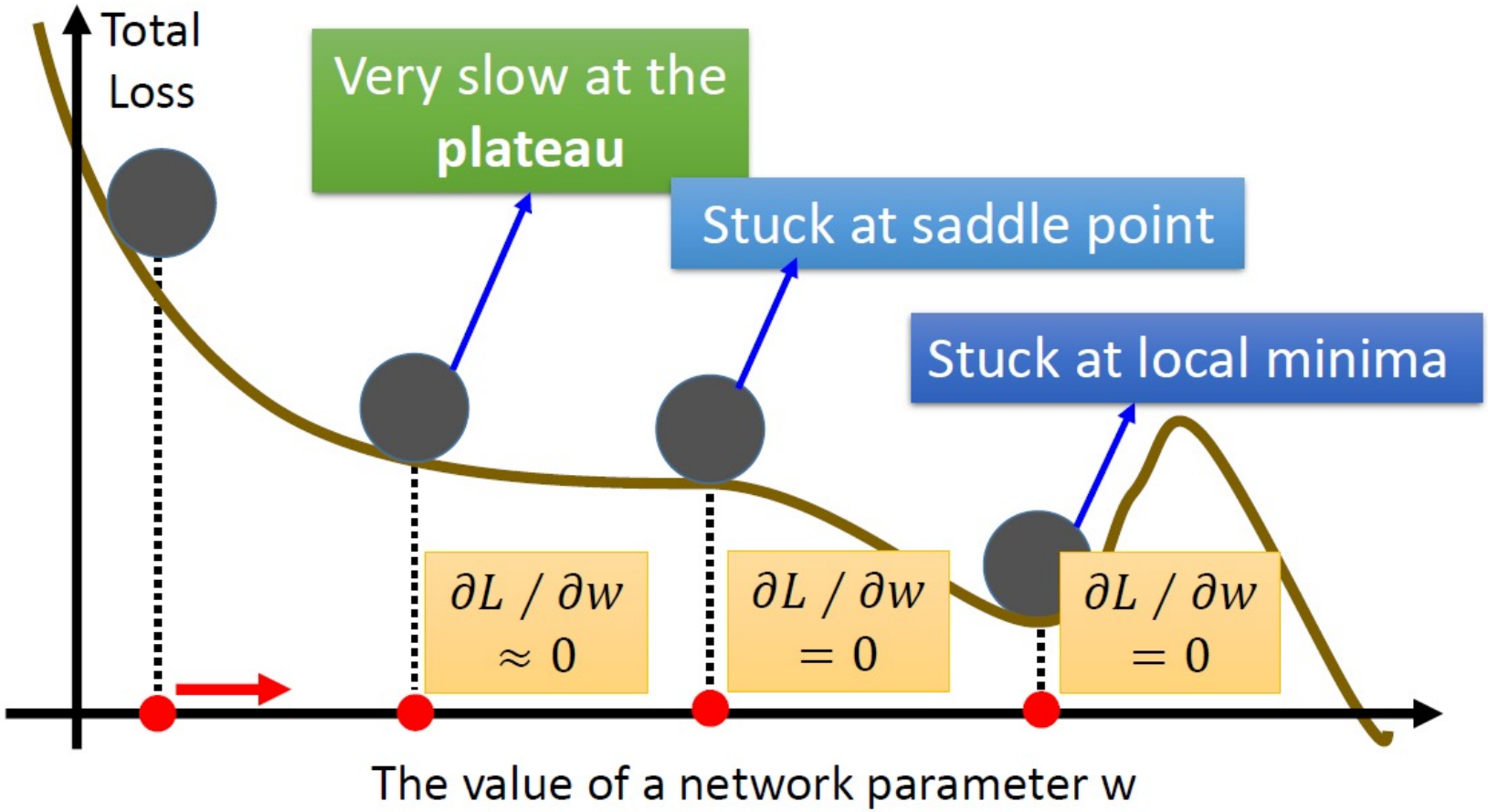
- Warning: by reducing the learning rate, you reduce the fluctuations, which can appear to make the loss drop suddenly. But this can come at the expense of long-run performance.





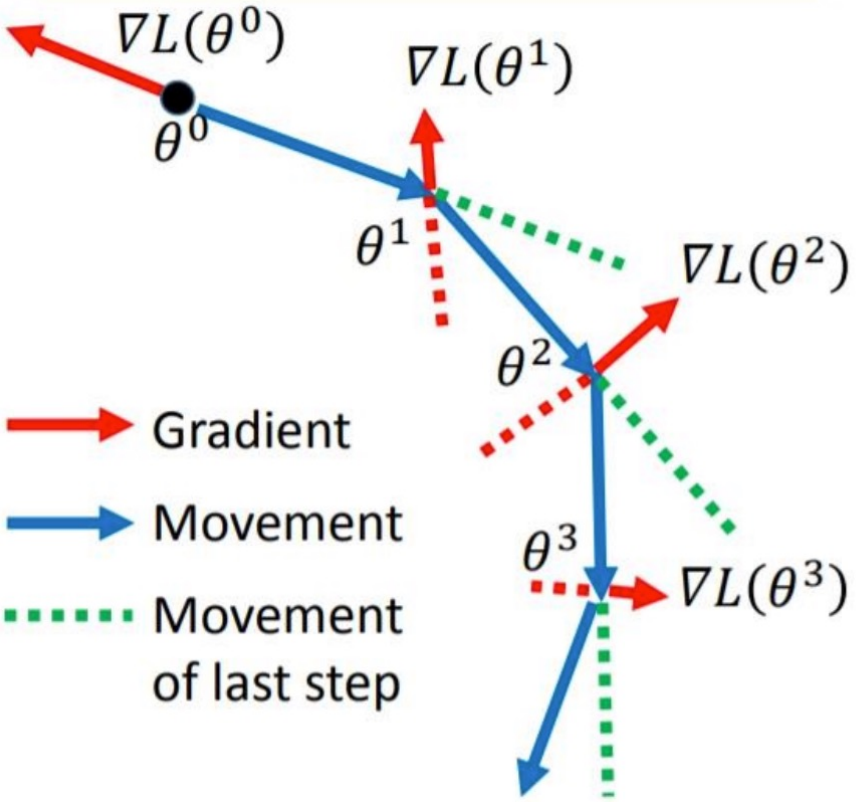
Momentum

- Reducing oscillations of movements



SGD with Momentum (SGDM)

Movement: movement of last step minus gradient at present



Start at point θ^0

Movement $v^0=0$

Compute gradient at θ^0

Movement $v^1 = \lambda v^0 - \eta \nabla L(\theta^0)$

Move to $\theta^1 = \theta^0 + v^1$

Compute gradient at θ^1

Movement $v^2 = \lambda v^1 - \eta \nabla L(\theta^1)$

Move to $\theta^2 = \theta^1 + v^2$

Movement not just based on gradient, but previous movement.

SGD with Momentum

v^i is actually the weighted sum of all the previous gradient: $\nabla L(\theta^0), \nabla L(\theta^1), \ldots \nabla L(\theta^{i-1})$

$v^0 = 0$

$v^1 = -\eta \nabla L(\theta^0)$

$v^2 = -\lambda \eta \nabla L(\theta^0) - \eta \nabla L(\theta^1)$

Physical Essence: Introducing inertia into gradient updates (conservation of momentum in physics)
Mathematical Essence: Exponentially weighted moving average of gradients

Start at point θ^0

Movement $v^0=0$

Compute gradient at θ^0

Movement $v^1 = \lambda v^0 - \eta \nabla L(\theta^0)$

Move to $\theta^1 = \theta^0 + v^1$

Compute gradient at θ^1

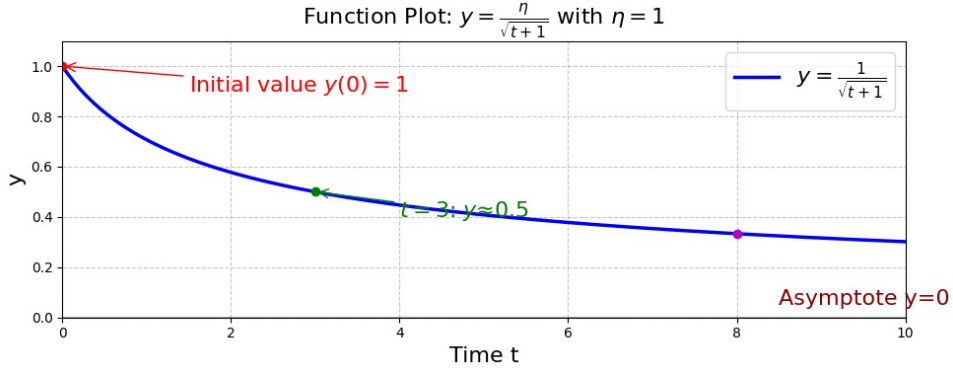
Movement $v^2 = \lambda v^1 - \eta \nabla L(\theta^1)$

Move to $\theta^2 = \theta^1 + v^2$

Movement not just based on gradient, but previous movement.

Adaptive Learning Rates

- **Popular & Simple Idea**: Reduce the learning rate by some factor every few epochs.
  - At beginning, we are far from the destination, so we use larger learning rate （大步流星）
  - After several epochs, we are close to the destination, so we reduce the learning rate （小步快跑）
  - e.g. 1/t decay: $\eta^t = \eta/\sqrt{t+1}$



Learning rate cannot be one-size-fits-all

⇒ Assign different learning rates to different parameters

Adagrad

- Divide the learning rate of each parameter by the root mean square of its previous derivatives

Vanilla Gradient descent

$$\eta_t = \frac{\eta}{\sqrt{t+1}} \qquad g_t = \frac{\partial L(\theta_t)}{\partial w}$$

$$w_{t+1} \leftarrow w_t - \eta_t g_t$$

Parameter dependent update

Adagrad

$$w_{t+1} \leftarrow w_t - \frac{\eta_t}{\sigma_t} g_t$$

σ^t: ***root mean square*** of the previous derivatives of w

Adagrad

$$w_1 \leftarrow w_0 - \frac{\eta_0}{\sigma_0} g_0 \qquad \sigma_0 = \sqrt{(g_0)^2}$$

$$w_2 \leftarrow w_1 - \frac{\eta_1}{\sigma_1} g_1 \qquad \sigma_1 = \sqrt{\frac{1}{2}[(g_0)^2 + (g_1)^2]}$$

$$w_3 \leftarrow w_2 - \frac{\eta_2}{\sigma_2} g_2 \qquad \sigma_2 = \sqrt{\frac{1}{3}[(g_0)^2 + (g_1)^2 + (g_2)^2]}$$

$$\vdots$$

$$w_{t+1} \leftarrow w_t - \frac{\eta_t}{\sigma_t} g_t \qquad \sigma_t = \sqrt{\frac{1}{t+1}\sum_{i=0}^{t}(g_i)^2}$$

Adagrad

- Divide the learning rate of each parameter by the root mean square of its previous derivatives.

$$w^{t+1} \leftarrow w^t - \frac{\eta^t}{\sigma^t} g^t$$

$$\eta^t = \frac{\eta}{\sqrt{t+1}}$$ 1/t decay

$$\sigma^t = \sqrt{\frac{1}{t+1}\sum_{i=0}^{t}(g^i)^2}$$

$$w^{t+1} \leftarrow w^t - \frac{\eta}{\sqrt{\sum_{i=0}^{t}(g^i)^2}} g^t$$

Contradiction?

$$\eta_t = \frac{\eta}{\sqrt{t+1}} \qquad g_t = \frac{\partial L(\theta_t)}{\partial w}$$

Vanilla Gradient descent

$$w^{t+1} \leftarrow w^t - \eta^t g^t$$

Larger gradient, larger step

Adagrad

$$w^{t+1} \leftarrow w^t - \frac{\eta}{\sqrt{\sum_{i=0}^{t}(g^i)^2}} g^t$$

Larger gradient, larger step

Larger gradient, smaller step

Intuitive Reason

$\sum_{i=0}^{t}(g_i)^2$ measures how surprise a new emerging feature is

反差

Super large value

g_0	g_1	g_2	g_3	g_4	……
0.001	0.001	0.003	0.002	0.1	……

$$\eta_t = \frac{\eta}{\sqrt{t+1}}$$

g_0	g_1	g_2	g_3	g_4	……
10.8	20.9	31.7	12.1	0.1	……

$$g_t = \frac{\partial L(\theta_t)}{\partial w}$$

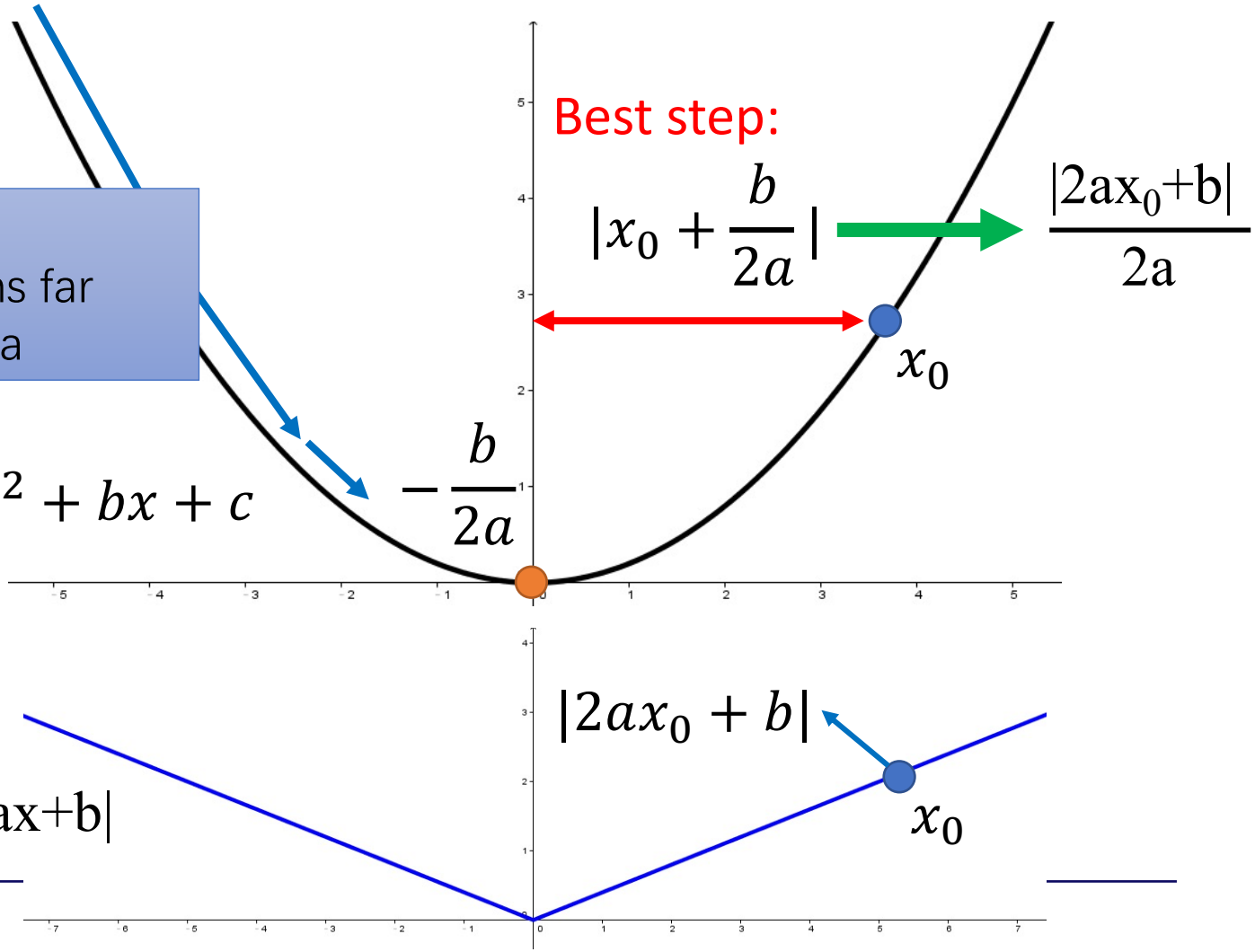
Super small value

$$w_{t+1} \leftarrow w_t - \frac{\eta}{\sqrt{\sum_{i=0}^{t}(g_i)^2}} g_t$$

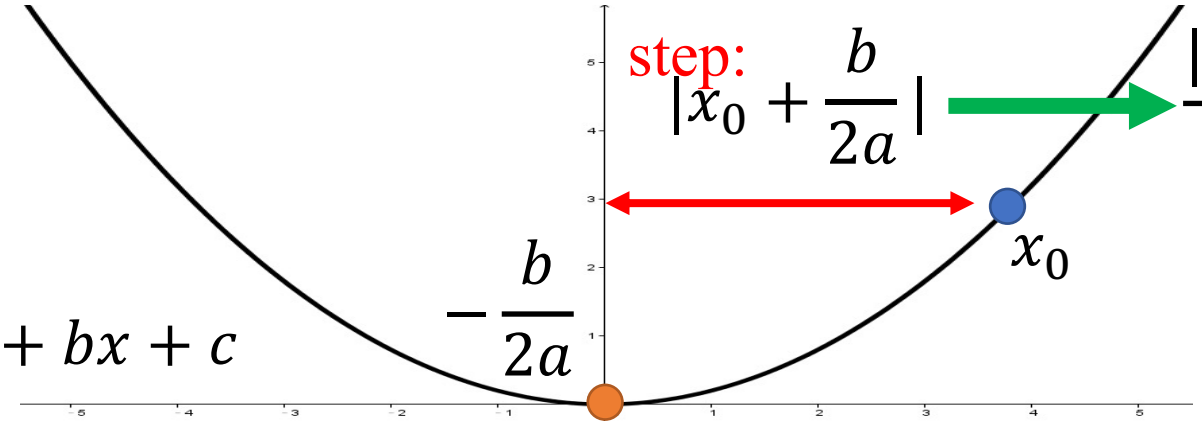
→ 造成反差的效果

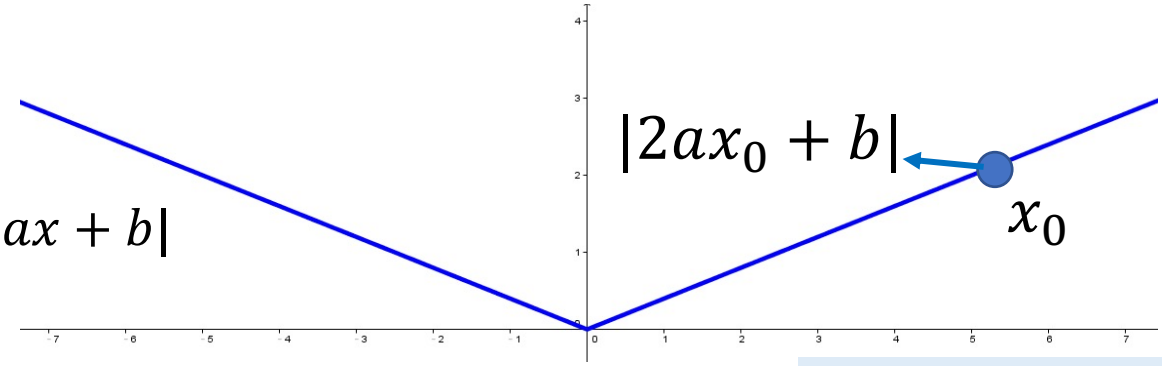
Larger gradient, larger steps?

Larger 1st order derivative means far from the minima



Second Derivative





$$\frac{\partial^2 y}{\partial x^2} = 2a$$

The best step is $\dfrac{|\text{First derivative}|}{\text{Second derivative}}$

$$w^{t+1} \leftarrow w^t - \frac{\eta}{\sqrt{\sum_{i=0}^{t}(g^i)^2}} g^t$$

The best step is:

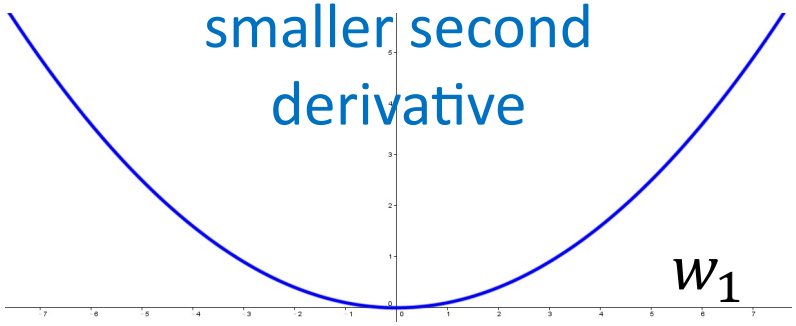
|First derivative|

Second derivative

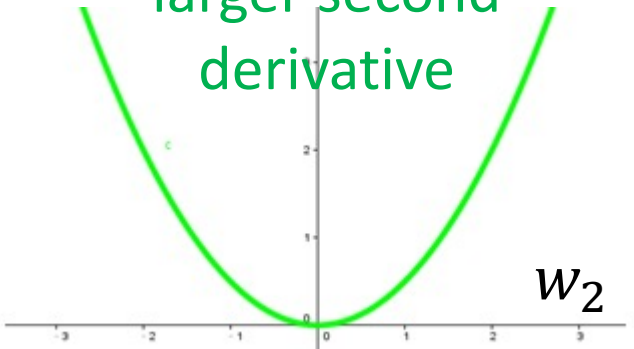
?

Use *first derivative* to estimate *second derivative*

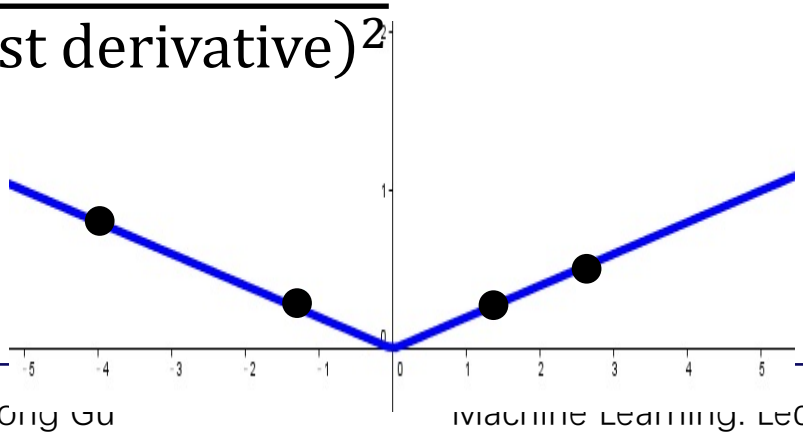
smaller second derivative

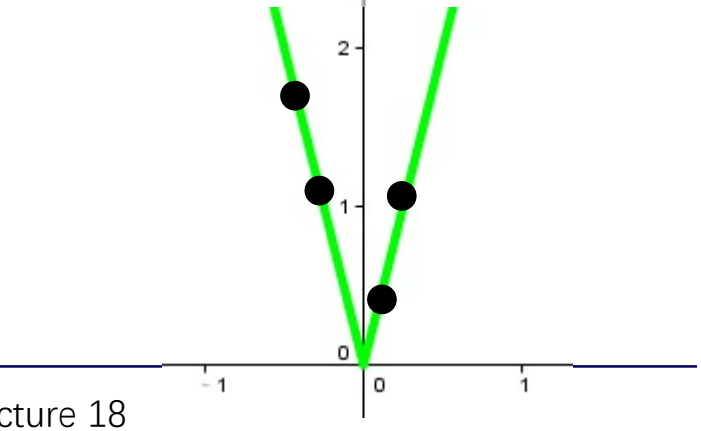


larger second derivative



$\sqrt{(\text{first derivative})^2}$





Adagrad: Advantages and Disadvantages



Advantages

1. As t continues to increase, the σ term constrains the gradients. This means Adagrad automatically adjusts the learning rate until convergence.
2. Well-suited for sparse gradients (e.g., in NLP/recommendation systems).

Disadvantages

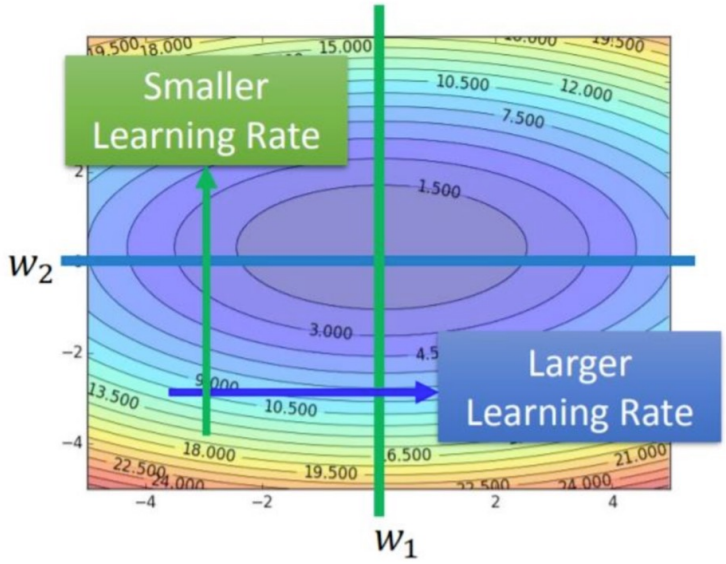
1. When $\sigma_t$becomes very large in later stages, gradients are constrained to approach 0, causing training to end prematurely.
2. Still requires manually setting a global learning rate η, which significantly impacts the training process.

RMSProp

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{v_t}} g_{t-1}$$

$$v_1 = {g_0}^2$$

$$v_t = \alpha v_{t-1} + (1-\alpha)(g_{t-1})^2$$



Exponential moving average (EMA) of squared gradients is not monotonically increasing

RMSProp: Advantages and Disadvantages



Advantages

1. Effectively addresses Adagrad's premature termination issue by using exponential moving average of squared gradients.
2. Well-suited for complex, non-convex error surfaces (e.g., training RNNs/LSTMs).

Disadvantages

1. Still requires manually setting a global learning rate η, which significantly impacts convergence.

Adadelta - another variant of Adagrad



- v_t: a leaky average of the second moment of the gradient
- $\Delta\theta_t$: a leaky average of the second moment of the change of parameters in the model itself.
- g'_{t-1}: rescaled gradient

$$\theta_t = \theta_{t-1} - g'_t \qquad g'_t = \frac{\sqrt{\Delta\theta_{t-1}+\epsilon}}{\sqrt{v_t+\epsilon}} \odot g_{t-1}$$

$$\Delta\theta_t = \alpha\, \Delta\theta_{t-1} + (1-\alpha) g'^{2}_{t-1}$$

$$v_t = \alpha v_{t-1} + (1-\alpha) g^{2}_{t-1}$$

Adam: Adaptive Moment Estimation

RMSProp + Momentum

- SGDM

$$\theta_t = \theta_{t-1} - \eta m_t$$
$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_{t-1}$$

- RMSProp

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{v_t}} g_{t-1}$$
$$v_1 = {g_0}^2$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(g_{t-1})^2$$

EMA mechanism

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t}+\varepsilon} \hat{m}_t$$

$$\hat{m}_t = \frac{m_t}{1-{\beta_1}^t}$$
$$\hat{v}_t = \frac{v_t}{1-{\beta_2}^t}$$

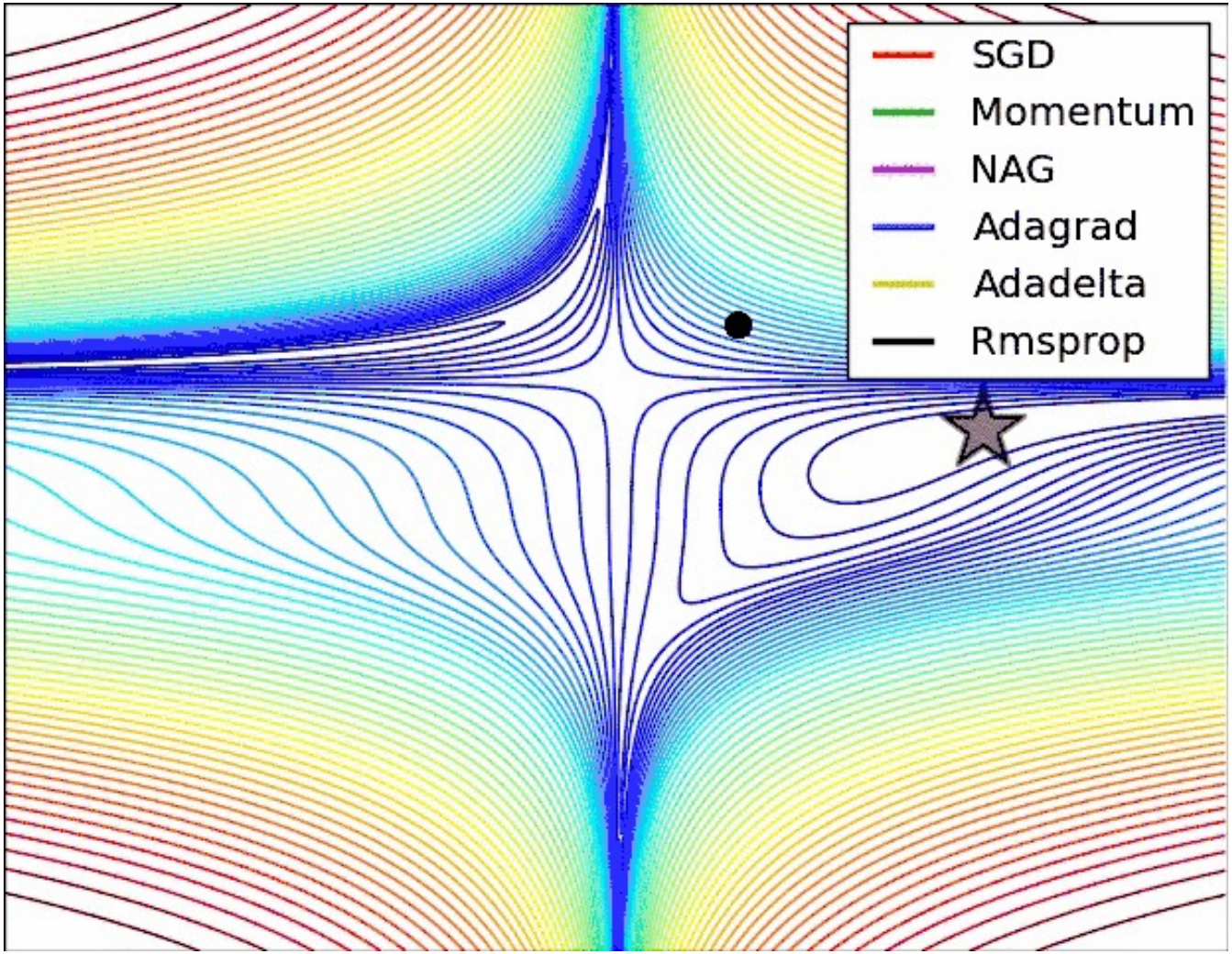
de-biasing

$$\beta_1 = 0.9$$
$$\beta_2 = 0.999$$
$$\varepsilon = 10^{-8}$$

Adam: Advantages

- Combines strengths of Adagrad, RMSprop, and Momentum:
- **Adaptive learning rate from Adagrad/RMSprop**
- **Momentum-based acceleration**
- Low memory footprint:
- Only requires two moving average vectors (first/second moments)
- Per-parameter trade-off adjustment:
- Balances historical vs. new gradient information via decay rates (β_1, β_2)
- Automatic bias correction for initialization effects
- Universal applicability:
- Current **gold-standard** for most deep learning tasks
- Robust across diverse architectures (CNNs, RNNs, Transformers)

Intuitive Comparison



Tips



- 如果数据是稀疏的，就用自适用方法，即 Adagrad, Adadelta, RMSprop, Adam。
- RMSprop, Adadelta, Adam 在很多情况下的效果是相似的。
- Adam 就是在 RMSprop 的基础上加了 bias-correction 和 momentum，
- 随着梯度变的稀疏，Adam 比 RMSprop 效果会好。
- 整体来讲，Adam 是最好的选择。

- SGD [Cauchy, 1847]
- SGD with momentum [Rumelhart, et al., Nature'86]

Adaptive learning rate

- Adagrad [Duchi, et al., JMLR'11]
- RMSProp [Hinton, et al., Lecture slides, 2013]
- Adam [Kingma, et al., ICLR'15]